Name _____

CSCI 332, Fall 2026

Exam 1 Practice 2

This exam consists of the following sections:

1. Stable Matching (25 points)
2. Algorithm Analysis (25 points)
3. Recursion (25 points)
4. Backtracking and 1D Dynamic Programming (25 points)

You may use a double sided 3x5 handwritten notecard of notes during the test but no other resources.

Good luck!

Section 1 (Stable Matching)

1. Given the matching $\{(h_4, s_1), (h_1, s_3), (h_3, s_4), (h_2, s_2), (h_5, s_5)\}$ and the preference lists below, select the unstable pairs for this matching from the list below by circling them.

Hospital	**1st choice**	**2nd choice**	**3rd choice**	**4th choice**	**5th choice**
h_1	s_1	s_2	s_5	s_3	s_4
h_2	s_4	s_5	s_1	s_3	s_2
h_3	s_1	s_5	s_2	s_3	s_4
h_4	s_3	s_2	s_4	s_1	s_5
h_5	s_1	s_4	s_2	s_5	s_3
Student	**1st choice**	**2nd choice**	**3rd choice**	**4th choice**	**5th choice**
s_1	h_2	h_4	h_3	h_1	h_5
s_2	h_4	h_2	h_5	h_1	h_3
s_3	h_4	h_3	h_2	h_1	h_5
s_4	h_3	h_4	h_5	h_1	h_2
s_5	h_4	h_1	h_5	h_3	h_2

   (a) (h_5, s_4)

   (b) (h_3, s_3)

   (c) (h_3, s_4)

   (d) (h_4, s_3)

   (e) (h_1, s_4)

Your friend proposes the following, simple algorithm for stable matching:

```
Gale-Shapley-Simplified(preference lists for n hospitals and n students)
  Let M be an empty matching
  While some hospital h is umatched:
      Let s be the first student on h's preference list that is not yet matched
      Add (h,s) to M
  Return M
```

2. What are the best-case and worst-case Θ runtimes for this algorithm in terms of n? You should give the simplest function possible.

3. Is the algorithm correct? If yes, simply write "yes". If not, give an instance for which Gale-Shapley-Simplified does not return a stable matching.

Section 2 (Algorithm Analysis)

4. Suppose you have algorithms with the following runtimes. (Assume these are exact running times as a function of the input size n, not asymptotic running times.) How much slower do these algorithms get when you double the input size? (You can find this by dividing the runtime on $2n$ by the runtime on n). You should simplify your answer as much as possible, including evaluating logarithms when possible.

   (a) $3n^2$

   (b) $\log_2 n$

   (c) 3^n

5. Order the following functions from asymptotically smallest to asymptotically largest by labeling the smallest with a 1 and increasing from there. If two functions are asymptotically equal (one is Θ of the other), give them the same number. For example, if the functions were n, n^2, and $3n + 4$, the labels should be 1, 2, 1.

   ___ $n \log n^3$
   ___ $n \log n^2$
   ___ $\sqrt{n}$
   ___ $n + \log n$
   ___ $n^{\log_3 2}$
   ___ $100^{100!}$
   ___ $\log_2 n$
   ___ 2^n
   ___ 3^n

6. Suppose you know that an algorithm has a worst-case runtime that is $O(n^2 \log n)$. For each of the following, decide whether it is definitely true, definitely false, or could be true or false and circle your choice.

   - The algorithm's worst-case runtime is $\Omega(n^2)$. T, F, T or F
   - The algorithm's worst-case runtime is $\Omega(n^3)$. T, F, T or F
   - The algorithm's best-case runtime is $O(n^2)$. T, F, T or F
   - The algorithm's worst-case runtime is $O(n^3)$. T, F, T or F
   - The algorithm's worst-case runtime is $\Theta(n^2 \log n)$. T, F, T or F

Section 3 (Recursion)

7. Given the following (meaningless) divide and conquer algorithm, what is the recurrence relation that describes its runtime? Indicate your answer by circling it. Notice the n is the number of rows and the number of columns in the square array A, not the total number of elements.

```
hi(square array A of with n rows and n columns):
  if n = 1:
      return sum(A)
  else:
      Let A_1,A_2,A_3,A_4 be the upper left, upper right, lower left, and lower right of A
      X_1 = hi(A_1 + A_4)
      X_2 = hi(A_2 + A_3)
      C = X_1 + X_2
      return C
```

A. $T(n) = 2T(n^2/2) + \Theta(n^2)$

B. $T(n) = T(n/2) + \Theta(n^2)$

C. $T(n) = 2T(n/2) + \Theta(n^2)$

D. $T(n) = 2T(n/2) + \Theta(n)$

E. $T(n) = 2T(n-2) + \Theta(n^2)$

The next two questions refer to the recurrence relation

$$T(n) = 3T(n/4) + \Theta(n^2).$$

8. What is the work done at each level of the recursion tree for this recurrence? Circle your answer.

A. $3^\ell + n^2$

B. $3^\ell \left(\frac{n}{4^\ell}\right)^2$

C. $\frac{4}{3}\ell^n$

D. $n^2/4 + 3^\ell$

E. $\frac{\ell \cdot n^2}{3}$

9. How many levels does the recursion tree have? Circle your answer.

A. $\log_4 n$

B. $\log_3 n$

C. n

D. $\sqrt[3]{n}$

E. $n \log_3 4$

The following questions refer to the problem of multiplying two n-digit numbers.

10. Below is a divide and conquer algorithm to multiply two n-digit numbers x and y based on the observation that, if a and b are the first and second halves of x and c, and d are the first and second halves of y respectively, then

$$x \cdot y = 10^n \cdot a \cdot c + 10^{n/2} \cdot (a \cdot d + b \cdot c) + b \cdot d.$$

```
Split-Multiply(length n integers x, y):
    # base case
    if n = 1:
        return _____

    # recursive case
    else:
        Split x into two halves a and b
        Split y into two halves c and d
        m = n/2
        e = Split-Multiply(a, c)
        f = Split-Multiply(b, d)
        g = Split-Multiply(b, c)
        h = Split-Multiply(a, d)
        return 10^(2m) · e + 10^m · (g + h) + f
```

What should be returned in the base case (the blank above)? Circle your answer.

A. 0

B. 1

C. $x + y$

D. $x \cdot y$

E. $x - y$

11. If $x = 1234$ and $y = 5678$, what are the values of a, b, c, and d when Split-Multiply$(x, y, 4)$ is called?

A. $a = 12$, $b = 34$, $c = 56$, $d = 78$

B. $a = 1$, $b = 234$, $c = 5$, $d = 678$

C. $a = 123$, $b = 4$, $c = 567$, $d = 8$

D. $a = 1234$, $b = 0$, $c = 5678$, $d = 0$

E. $a = 0$, $b = 1234$, $c = 0$, $d = 5678$

12. What is the runtime of Split-Multiply as a function of n, the number of digits in the input numbers x and y?

A. $\Theta(n)$

B. $\Theta(n \log n)$

C. $\Theta(n^2)$

D. $\Theta(n^{\log_2 3})$

E. $\Theta(2^n)$

Section 4 (1D Dynamic Programming)

Having spied some snow on the mountains, you are thinking about ski season. You will ski a ski jumping course with n jumps, each evenly spaced one meter apart. The height that each jump will launch you is stored in an array $H[1..n]$. However, each jump will also launch you 2 meters in horizontal distance, meaning that if you take a jump, you must skip the next jump. You can always choose to skip a jump. You would like to accumulate the largest total height jumped by choosing a set of jumps to take.

(a) Suppose the jump heights are given by the following array (indices noted above):

	1	2	3	4	5	6
H	2	4	4	1	3	1

What is the optimal set of jumps to take and what is the total height?

(b) To solve this problem, you decide on the following English definition of the subproblems.

The subproblem. Let $MaxHeight(i)$ be the maximum total height you can jump, assuming you only consider jumps i through n.

What is $MaxHeight(4)$ for the above H?

Next, we think about the recursive relationship between subproblems. You notice that you have two choices: either take the ith jump, or don't.

(c) **The recursive definition.**

$$MaxHeight(i) = \begin{cases} _____ & if\ i_____ \\ _____ & if\ i_____ \\ _____ & if\ i_____ \\ _____ & if\ i_____ \\ _____ & if\ i_____ \end{cases}$$

How to memoize? We can store solutions to the subproblems in an array. What direction should we fill it (left to right or right to left)?

(d) **Write the full algorithm.** Using the above information, finish the pseudocode to find the maximum amount of air you can get on the ski jumps, given $H[1..n]$.

```
SkiJump(H[1..n]):
  Let array M be an array of length n
  # do we have any base cases?




  # how should we fill in the rest of the array?




  # what should we return to get the final answer?
```